

Software Architecture Specification Carbon Credit Exchange Trading System Proof Of Concepts AI-Generated Architecture and tradeoff's (by Claude)

1. System Summary

The Carbon Credit Exchange Trading System is a digital marketplace that converts verified sustainability activities performed by Sellers (SMEs) into tradable carbon credits, which Buyers (SMEs) can discover and purchase under Admin-defined rules. The system's primary objectives are to automate the lifecycle of a carbon credit submission, verification, activation, listing, purchase, transfer, and logging while ensuring transparency, compliance, and auditability at each stage. The scope covers three user classes (Seller, Buyer, Admin) and the workflows connecting them: sustainability action submission and verification, credit calculation and issuance, marketplace listing and ranking, purchase processing with compliance checks, receipt generation, portfolio/balance monitoring, and administrative oversight (roles, pricing policy, monitoring, reporting).

2. Assumptions

ID	Assumption
AS-01	The SRS does not specify a deployment environment (cloud, on-premise, containerized, etc.); Section 14 therefore presents a logical deployment view rather than a concrete technology-bound deployment.
AS-02	The SRS does not name a specific persistence technology; Section 8 describes data ownership and persistence responsibilities without assuming a database product or paradigm.
AS-03	The SRS does not specify the mechanism of the "broker ranking" function beyond its ranking criteria (FR-B05) and failure behavior (FR-UC14); it is treated as a distinct logical capability responsible for ranking and re-ranking listings.
AS-04	The SRS does not specify how "network or database error" (FR-UC10, FR-UC13) is detected; this is treated as an operational error condition surfaced to the Listing capability, not a specific infrastructure component.
AS-05	The SRS does not specify the payment processing mechanism internals beyond compliance/eligibility/limit/rule validation (FR-B07–FR-B10) and rollback/failure behavior (FRUC-01, FR-UC08); payment processing is treated as a logical capability within the Purchase workflow rather than a named external payment gateway, since no external payment service is explicitly stated.
AS-06	"Transaction-safe methods" (NFR06) is interpreted as an architectural requirement for concurrency control on shared, contended state (credit stock, listing status) rather than a specific locking algorithm.

These are the only assumptions required; all other architectural decisions trace directly to explicit SRS content.

3. Selected Architecture Style

Selected Style: Layered (N-Tier) Architecture combined with Modular, Role-Oriented Service Decomposition

Justification:

The SRS explicitly defines three distinct actor roles (Seller, Buyer, Admin) each with a bounded, non-overlapping set of functional requirements (FR-B*, FR-A*, FR-UC*). This naturally drives a decomposition into role-aligned functional modules (Account/Access, Listing, Purchase, Verification, Credit Issuance, Reporting/Monitoring), which is best expressed as a **modular, service-oriented logical decomposition**.

At the same time, cross-cutting requirements – secure communication (NFR04), auditability via logging (NFR05), extensibility without structural change (NFR01–NFR03), and transaction safety (NFR06) – are best satisfied by separating **presentation/interface concerns**, **business/workflow logic**, and **data/persistence concerns** into distinct layers, since this isolates change (extensibility) and isolates trust boundaries (security).

No event-driven, microkernel, or peer-to-peer style is explicitly justified by the SRS: there is no explicit requirement for independent deployability, asynchronous messaging infrastructure, or plugin-based extension loading – only that new roles/rules be addable "with minimal code refactoring" (NFR02) and "via configuration" (NFR03), which a well-modularized layered system satisfies without additional architectural complexity. A layered + modular approach is therefore the minimal style that satisfies all stated functional and quality requirements without over-engineering beyond what the SRS supports.

4. Architectural Drivers

Functional Drivers

- Account management and secure access for three distinct roles (FR-B01–03, FR-A01–02, FR-A16-18)
- Seller sustainability action submission and Admin-driven verification/approval workflow (FR-A03–08, CON04)
- Carbon credit issuance with UUID and expiry assignment (FR-A06–08)
- Credit listing with minimum-price enforcement and draft/retry handling (FR-UC10–13, CON03)
- Credit ranking by weighted score (FR-B05, FR-UC14)
- Purchase workflow with multi-stage validation, rollback, reservation, and confirmation (FR-B06–13, FR-UC01–09)
- Portfolio/balance/expiry monitoring for Buyers (FR-B14-16)
- Admin monitoring, pricing policy, and reporting (FR-A09-15)
- Transaction logging for auditability (Product Function: Transaction logging; NFR05)

Quality Attribute Drivers

- **Extensibility (NFR01–03):** Drives modular decomposition by role/capability and externalized, configuration-driven rule definitions, so new roles or rules do not require structural rework.
- **Security (NFR04):** Drives a distinct boundary/interface layer enforcing HTTPS transport and token-based authentication for every inbound request, independent of business logic.
- **Reliability/Auditability (NFR05):** Drives a dedicated logging/audit capability that records a UUID for every transaction, decoupled from workflow components so it cannot be bypassed.
- **Reliability - Concurrency (NFR06):** Drives transaction-safe (contention-controlled) access to shared mutable state (credit stock, listing rank/status) during verification and trading.
- **Performance (NFR07–08):** Drives bounded-latency design for the Purchase workflow (≤ 1 minute) and the Listing publication workflow (≤ 1 minute), influencing that these workflows be direct and synchronous rather than introducing unnecessary intermediate steps.

5. Requirements-to-Architecture Traceability

Requirement ID	Requirement	Architectural Component(s)	Justification
FR-B01–B03	Buyer registration, login, secure account management	Access & Account Management Component	Encapsulates identity/account lifecycle for all roles
FR-A01–A02	Admin authentication, role management	Access & Account Management Component	Same account/identity capability serves all roles
FR-A16–A18	Approve/suspend/delete user accounts	Access & Account Management Component	Account lifecycle governance
FR-B04	Browse verified credits	Credit Listing & Marketplace Component	Presents available inventory to Buyers
FR-B05, FR-UC14	Weighted ranking of listings	Listing Ranking Component	Distinct ranking responsibility with independent failure mode
FR-UC10–UC13, CON03	Listing submission, draft-on-error, minimum price rejection	Credit Listing & Marketplace Component	Owns listing creation/validation lifecycle
FR-B06–B10	Purchase, compliance/eligibility/limit/rule validation	Purchase & Transaction Processing Component	Central purchase workflow orchestration
FRUC-01, FR-UC02,	Rollback, reservation, unavailability handling, cancellation, payment failure	Purchase & Transaction	All purchase state-machine behaviors

FR-UC06–UC09		Processing Component	
FR-UC03–UC04	Update buyer portfolio, decrease seller stock	Purchase & Transaction Processing Component ↔ Credit Portfolio Component, Credit Listing & Marketplace Component	Cross-component state updates following successful purchase
FR-B11–B13, FR-UC05, FR-UC09	Digital receipt generation with UUID, confirmation display, retry	Receipt & Confirmation Component	Isolated, retryable responsibility distinct from purchase orchestration
FR-B14–B16	View purchases, monitor balance/expiry	Credit Portfolio Component	Buyer-owned view of held credits
FR-A03–A05, CON04	Review/approve/reject seller submissions with evidence	Verification & Approval Component	Encapsulates Admin evidence-review workflow
FR-A06–A08, CON01, CON05	Issue credits with UUID/expiry upon approval	Credit Issuance Component	Distinct issuance responsibility triggered by verification outcome
FR-A09–A10, CON03	Define/update minimum selling price	Pricing Policy Component	Central, reusable policy source for listing validation
FR-A11–A13	Monitor trading data, transactions, system health	Monitoring & Analytics Component	Aggregated administrative oversight
FR-A14	Configure automatic expiry alerts	Monitoring & Analytics Component	Alerting tied to monitored credit state (CON02)
FR-A15	Export analytical reports	Monitoring & Analytics Component	Reporting derived from monitored data
Transaction logging (2.1)	Log all transactions	Audit & Logging Component	Cross-cutting, applies to every transactional workflow
NFR04	HTTPS + JWT	Access & Account Management Component / API Boundary	Enforced at the system's inbound boundary
NFR05	Unique ID+ logging for auditability	Audit & Logging Component	Dedicated cross-cutting capability

NFR06	Transaction-safe operations	Purchase & Transaction Processing Component, Credit Listing & Marketplace Component	Concurrency control at points of shared state contention
NFR01–03	Extensibility	All components (modular boundaries), Pricing/Rule configuration	Modular decomposition + externalized rule/price configuration
CON02	Depreciation after 70% validity	Credit Portfolio Component, Monitoring & Analytics Component	Expiry/depreciation is a monitored credit-state attribute

6. Logical Architectural Components

6.1 Access & Account Management Component

- **Primary Responsibility:** Registration, authentication, secure session/account management, and role/account lifecycle governance for Buyers, Sellers, and Admins.
- **Inputs:** Registration data, login credentials, account management requests, Admin role/account actions.
- **Outputs:** Authenticated session tokens, account status, role assignments.
- **Public Interfaces:** RegisterAccount, Authenticate, ManageAccount, ManageUserRoles, ApproveAccount, SuspendAccount, DeleteAccount.
- **Dependencies:** Audit & Logging Component (for account action logging).

6.2 Verification & Approval Component

- **Primary Responsibility:** Review Seller-submitted sustainability actions and supporting evidence; approve or reject submissions.
- **Inputs:** Submitted sustainability actions, supporting documentation (CON04), Admin review decisions.
- **Outputs:** Approval or rejection outcome, verified action record.
- **Public Interfaces:** SubmitSustainabilityAction, ReviewSubmission, ApproveSubmission, RejectSubmission.
- **Dependencies:** Credit Issuance Component (triggered on approval), Audit & Logging Component.

6.3 Credit Issuance Component

- **Primary Responsibility:** Calculate and issue verified carbon credits, assigning UUID and expiry period upon approval; enforce minimum accumulation threshold prior to submission eligibility.
- **Inputs:** Approved verification outcome, sustainability action data, minimum credit accumulation rule (CON01).

- **Outputs:** Verified, uniquely identified carbon credit records with expiry metadata.
- **Public Interfaces:** IssueCredit, AssignExpiry.
- **Dependencies:** Verification & Approval Component, Audit & Logging Component.

6.4 Credit Listing & Marketplace Component

- **Primary Responsibility:** Enable Sellers to list verified/active credits for sale; validate listing price against policy; handle draft-on-error and retry; present available listings to Buyers.
- **Inputs:** Seller listing submissions, credit availability/status, minimum price policy.
- **Outputs:** Published listings, draft listings, rejection/revision prompts.
- **Public Interfaces:** SubmitListing, RetryListing, GetAvailableListings, DecrementStock.
- **Dependencies:** Pricing Policy Component, Listing Ranking Component, Credit Issuance Component (credit validity), Audit & Logging Component.

6.5 Listing Ranking Component

- **Primary Responsibility:** Compute weighted ranking of active listings based on validity, price, and quantity; keep listings visible-but-unranked when ranking fails, until recalculation succeeds.
- **Inputs:** Active listing data (validity, price, quantity).
- **Outputs:** Ranked (or pending-rank) listing order.
- **Public Interfaces:** RankListings, RecalculateRanking.
- **Dependencies:** Credit Listing & Marketplace Component.

6.6 Pricing Policy Component

- **Primary Responsibility:** Define and update the platform's minimum selling price per credit; serve as the authoritative source for listing price validation.
- **Inputs:** Admin-defined/updated minimum price.
- **Outputs:** Current minimum price policy.
- **Public Interfaces:** SetMinimumPrice, UpdateMinimumPrice, GetMinimumPrice.
- **Dependencies:** None (policy source consumed by other components).

6.7 Purchase & Transaction Processing Component

- **Primary Responsibility:** Orchestrate the Buyer purchase workflow: compliance, eligibility, purchase-limit, and rule-conformance validation; payment handling; reservation; rollback on failure; cancellation handling; stock decrement and portfolio update on success.
- **Inputs:** Buyer purchase requests, selected listing, compliance/eligibility/limit/rule data, payment outcome.
- **Outputs:** Purchase confirmation or failure notice, reserved credit records, updated stock/portfolio triggers.
- **Public Interfaces:** InitiatePurchase, ValidatePurchase, ReserveCredits, ConfirmPurchase, CancelPurchase, RollbackTransaction.
- **Dependencies:** Credit Listing & Marketplace Component, Pricing Policy Component (rule conformance), Credit Portfolio Component, Receipt & Confirmation Component, Audit & Logging Component.

6.8 Receipt & Confirmation Component

- **Primary Responsibility:** Generate a digital receipt (with UUID and transaction details) following a successful purchase; display confirmation; retry receipt generation on error.
- **Inputs:** Successful purchase/transaction data.
- **Outputs:** Digital receipt, confirmation message.
- **Public Interfaces:** GenerateReceipt, RetryReceiptGeneration, DisplayConfirmation.
- **Dependencies:** Purchase & Transaction Processing Component, Audit & Logging Component.

6.9 Credit Portfolio Component

- **Primary Responsibility:** Maintain the Buyer's purchased-credit portfolio; provide balance and expiry-status monitoring; reflect depreciation status per validity policy (CON02).
- **Inputs:** Confirmed purchase updates, credit expiry/validity data.
- **Outputs:** Portfolio view, balance status, expiry/depreciation status.
- **Public Interfaces:** GetPortfolio, GetBalance, GetExpiryStatus.
- **Dependencies:** Purchase & Transaction Processing Component.

6.10 Monitoring & Analytics Component

- **Primary Responsibility:** Provide Admin visibility into trading data, transactions, and system health; configure automatic expiry alerts; export analytical reports.
- **Inputs:** Trading/transaction/system-health data, expiry alert configuration requests.
- **Outputs:** Monitoring dashboards/data, expiry alerts, exported reports.
- **Public Interfaces:** MonitorTradingData, MonitorTransactions, MonitorSystemHealth, ConfigureExpiryAlerts, ExportReports.
- **Dependencies:** Audit & Logging Component, Credit Portfolio Component (expiry data), Purchase & Transaction Processing Component (transaction data).

6.11 Audit & Logging Component

- **Primary Responsibility:** Generate and persist a verifiable UUID and log entry for every transaction across the system, supporting auditability.
- **Inputs:** Transaction/action events from all other components.
- **Outputs:** Persisted, verifiable log records.
- **Public Interfaces:** LogTransaction, GenerateAuditUUID.
- **Dependencies:** None (consumed by all other components).

7. Component Interactions

Control Flow:

The Access & Account Management Component authenticates every inbound Buyer/Seller/Admin request before it is routed to a role-specific workflow component. Seller-originated flows proceed: Verification & Approval → Credit Issuance → Credit Listing & Marketplace → Listing Ranking. Buyer-originated flows proceed: Credit Listing & Marketplace (browse/rank) → Purchase & Transaction Processing → Receipt & Confirmation and Credit Portfolio. Admin-originated flows reach Pricing Policy, Verification

& Approval, Monitoring & Analytics, and Access & Account Management (role/account governance) directly.

Data Flow:

Verified sustainability action data flows from Verification & Approval into Credit Issuance, producing credit records consumed by Credit Listing & Marketplace. Listing data flows into Listing Ranking for scoring and back into the marketplace view. Purchase requests flow into Purchase & Transaction Processing, which reads listing/pricing data, writes reservation and stock-decrement updates back to Credit Listing & Marketplace, writes portfolio updates to Credit Portfolio, and emits transaction data to Receipt & Confirmation. All components emit event/action data to Audit & Logging. Monitoring & Analytics reads aggregated transaction, portfolio, and log data for reporting.

Communication:

Interactions are modeled as synchronous request/response calls between components via their public interfaces, consistent with the bounded-latency requirements (NFR07–08) for purchase and listing workflows. The Audit & Logging Component receives asynchronous-style notification calls from all other components so that logging does not block primary workflow completion, while remaining consistent (every transaction is logged, per NFR05).

8. Data Management Strategy

Data Ownership:

Each logical component owns the data it is primarily responsible for: Access & Account Management owns account/role/session data; Verification & Approval owns submission/evidence records; Credit Issuance owns credit identity (UUID, expiry) records; Credit Listing & Marketplace owns listing records (including draft state); Pricing Policy owns the minimum-price policy value; Purchase & Transaction Processing owns transaction/reservation state; Credit Portfolio owns Buyer holdings; Audit & Logging owns the immutable transaction/action log.

Persistence Strategy:

The SRS does not specify a database technology; persistence is therefore described only as a requirement, not a technology choice. All transactional and credit-state data (listings, purchases, credit records) must be persisted using transaction-safe methods (NFR06) to prevent race conditions during concurrent verification and trading operations. Draft listings (FR-UC10) and rolled-back transactions (FRUC-01) require that intermediate/failed states be persisted or discarded consistently rather than left ambiguous.

Data Flow:

Credit records move from Credit Issuance → Credit Listing & Marketplace → (upon purchase) Credit Portfolio, with each transition recorded by Audit & Logging. Pricing Policy data is read (not owned) by Credit Listing & Marketplace at listing-validation time.

Storage Responsibilities:

Each component is responsible for the integrity and consistency of the data it owns; no shared, uncontrolled mutable state is implied by the SRS. Cross-component data updates (e.g., stock decrement and portfolio update following purchase, FR-UC03–UC04) must occur consistently as part of the same logical transaction outcome to satisfy NFR06.

9. External System Interactions

The SRS does not explicitly name any external system (e.g., no named payment gateway, identity provider, or notification service is stated). The only externally-oriented dependencies explicitly stated are:

Dependency (from SRS Section 2.2)	Nature
Verified sustainability evidence	External input supplied to the Verification & Approval Component (documentation supporting a submission, per CON04)
Standard environmental metrics	External reference data used in carbon credit calculation
Minimum market price defined by the platform	Internally governed (Pricing Policy Component), not an external system

No payment gateway, identity provider, or notification service is named in the SRS; the Purchase workflow's payment/compliance validation (FR-B07–B10) and buyer notification on payment failure (FR-UC08) are therefore modeled as internal logical capabilities of the Purchase & Transaction Processing Component, per Assumption AS-05, rather than as external system integrations.

10. Architectural Quality Considerations

- **Modularity:** Each functional area (account, verification, issuance, listing, ranking, pricing, purchase, receipt, portfolio, monitoring, audit) is realized as a separate component with explicit public interfaces, directly supporting NFR01–03.
- **Maintainability:** Role-aligned component boundaries mean a change to one workflow (e.g., purchase validation rules) does not require structural change elsewhere, satisfying NFR01.
- **Extensibility:** New actor roles can be introduced primarily through the Access & Account Management Component's role model (NFR02); new rules can be introduced through the Pricing Policy Component and purchase-validation configuration rather than code changes (NFR03).
- **Security:** Centralizing authentication in the Access & Account Management Component and requiring HTTPS/JWT (NFR04) at the system boundary ensures every request is authenticated before reaching business logic.
- **Reliability/Auditability:** The dedicated Audit & Logging Component ensures every transaction receives a verifiable UUID and log entry (NFR05), independent of which workflow component initiated it.
- **Reliability (Concurrency):** Isolating shared, contended state (credit stock in Credit Listing & Marketplace, reservation state in Purchase & Transaction Processing) behind transaction-safe interfaces addresses NFR06.
- **Performance:** Direct, synchronous interaction paths for Purchase (FR-B06–13) and Listing (FR-UC10–13) workflows, without unnecessary intermediate components, support the one-minute completion targets (NFR07–08).

No scalability, interoperability, or availability requirements are explicitly stated in the SRS; these attributes are therefore not addressed as architectural drivers, per the constraint against introducing unsupported quality attributes.

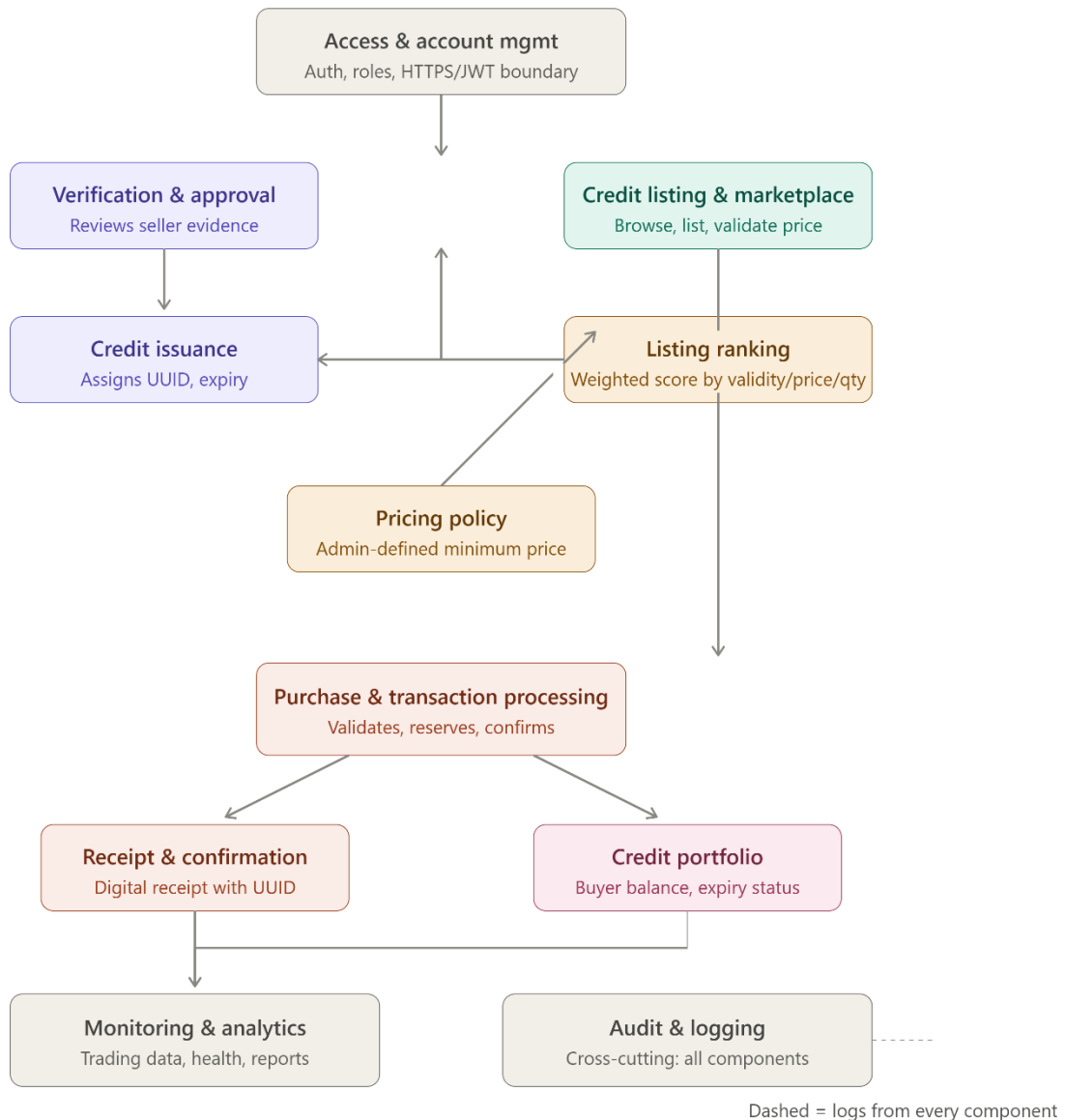
11. Architectural Decisions and Trade-offs

Decision	Rationale	Advantages	Potential Disadvantages
Decompose system by role-aligned functional components rather than a single monolithic module	FR-B*, FR-A*, FR-UC* map cleanly to distinct responsibilities	Directly satisfies NFR01–03; clear traceability	More inter-component interfaces to define and keep consistent
Separate Listing Ranking from Credit Listing & Marketplace	FR-UC14 specifies ranking can fail independently of listing itself	Isolates a distinct failure mode (unranked-but-visible) without affecting listing availability	Adds an additional component boundary and interface
Separate Receipt & Confirmation from Purchase & Transaction Processing	FR-UC09 specifies receipt generation can be retried independently of the purchase outcome	Purchase can be marked successful (FR-UC05) even while receipt generation retries, matching stated behavior	Requires the two components to coordinate final transaction state
Centralize minimum price in a dedicated Pricing Policy Component	FR-A09–10 and CON03 both reference a single, admin-governed minimum price used across listing validation	Single source of truth avoids inconsistent price enforcement	Adds a dependency that Credit Listing & Marketplace must query at listing time
Model payment/compliance validation as an internal capability, not an external gateway	No external payment system is named in the SRS	Avoids introducing unsupported external dependencies	If an external payment processor exists in reality but was omitted from the SRS, this component boundary would need revision
Dedicated, independently-invoked Audit & Logging Component	Transaction logging is listed as a distinct product function and NFR05 applies system-wide	Guarantees consistent, non-bypassable logging across all workflows	Every component must remember to invoke it; discipline-dependent without enforcement mechanism specified in the SRS

12. Architectural Risks and Limitations

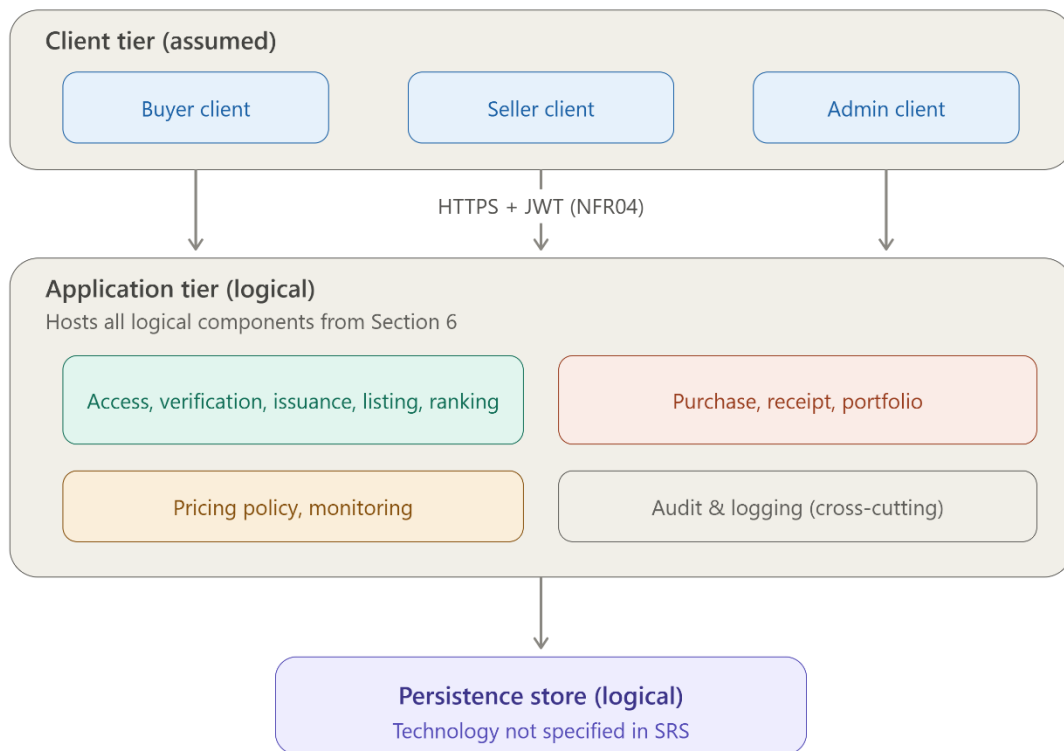
- **Undefined persistence mechanism:** Because no database technology is specified, the transaction-safety requirement (NFR06) can only be addressed at the logical/interface level; actual concurrency guarantees depend on implementation choices outside this SRS's scope.
- **Undefined payment mechanism:** FR-B07–B10 and FR-UC08 imply payment validation and failure handling, but no payment process detail is given; the boundary between "internal" and potential future "external" payment integration is a risk area if requirements are later clarified.
- **Ambiguous "network or database error" detection (FR-UC10, FR-UC13):** The SRS does not specify how such errors are detected or by which component, which may require future refinement.
- **Undefined rule format for "rule conformance" (FR-B10) and "configuration-based rules" (NFR03):** No schema or structure for admin-defined rules is specified, which limits precision in the Purchase & Transaction Processing and Pricing Policy component interfaces.
- **No explicit scalability, availability, or interoperability requirements:** The architecture does not address these attributes because the SRS does not require them; if real-world deployment needs them, the architecture would require extension.
- **Deployment environment unspecified:** Section 14 is necessarily a logical, assumption-based view given the absence of deployment requirements in the SRS.

13. Mermaid Component Diagram



14. Mermaid Deployment Diagram

Note: The SRS does not specify a deployment environment, hosting model, or infrastructure. The diagram below therefore presents a **logical deployment view only** grouping components by client-side vs. server-side responsibility, per Assumption AS-01 and introduces no specific technology, cloud provider, or middleware.



15. Conclusion

The proposed layered, role-oriented modular architecture satisfies the Carbon Credit Trading Exchange Platform SRS by directly tracing every functional requirement (FR-B*, FR-A*, FR-UC*) to a bounded logical component with clear responsibilities, inputs, outputs, and interfaces (Sections 5-6). The decomposition into eleven components spanning access control, the Seller-side verification-to-issuance pipeline, the Buyer-side listing-to-purchase pipeline, and cross-cutting pricing, monitoring, and audit capabilities satisfies the extensibility requirements (NFR01-03) by isolating change to single components and externalizing rule/price configuration. The centralized Access & Account Management boundary and mandated HTTPS/JWT (NFR04) satisfy the security requirement at the system's single point of entry. The dedicated Audit & Logging component, invoked by every workflow, satisfies the auditability requirement (NFR05) by guaranteeing a UUID and log entry per transaction. Isolating shared mutable state (credit stock, reservation status) behind transaction-safe interfaces addresses the concurrency requirement (NFR06), and direct, synchronous interaction paths for the purchase and listing workflows support the one-minute performance targets (NFR07–08). All constraints (CON01-05) and assumptions (A-01-05) from the SRS are respected without introducing unstated business functionality, technologies, or external systems, and the six assumptions documented in Section 2 mark the only points where the SRS's silence required interpretation.